



Connect til Siemens PLC

*Python **snap7** biblioteket*

Læringsmål

- ✓ Forstå og anvende `python-snap7` biblioteket til PLC-kommunikation
- ✓ Oprette forbindelse til en Siemens S7-PLC via Ethernet
- ✓ Læse data (Bool, Int, Real) fra en Data Block i PLC'en
- ✓ Skrive data tilbage til PLC'en
- ✓ Implementere betingede handlinger og fejlhåndtering



Indhold

1	Introduktion	2
1.1	Installation	2
2	Opret forbindelse til PLC	3
3	Læse data fra PLC (db_read)	4
4	Skrive data til PLC (db_write)	5
5	Betingede handler (if-sætning)	6
6	Fejlhåndtering (try / except / finally)	7
7	Oversigt over snap7 funktioner	8
8	Øvelse	8

1 | Introduktion

Siemens S7-PLC'er er et af de mest udbredte PLC-systemer inden for industriel automation. For at kommunikere med dem fra Python bruger vi biblioteket **python-snap7**, som er en Python-wrapper til det åbne **snap7**-bibliotek skrevet i C++.

Hvad er snap7?

snap7 er et gratis, open-source bibliotek der implementerer Siemens S7 kommunikationsprotokollen (S7comm). Det understøtter S7-200, S7-300, S7-400, S7-1200 og S7-1500 serien.

1.1 Installation

Installer **python-snap7** via pip:

Terminal — installation

```
1 pip install python-snap7
```

Vigtigt: PLC-opsætning i TIA Portal

Før Python kan tale med PLC'en skal følgende aktiveres i **TIA Portal**:

- *PUT/GET communication* skal være tilladt under *Protection*
- *Optimized block access* skal være **slået fra** i Data Block egenskaber
- IP-adressen på PLC'en skal kendes

2 | Opret forbindelse til PLC

Den grundlæggende forbindelsesprocedure kræver PLC'ens **IP-adresse**, **rack**-nummer og **slot**-nummer. For S7-1200/1500 er rack typisk 0 og slot 1. Nogle opsætninger kræver en specifik port (standard er 102, men kan ændres til f.eks. 1102).

Kode 1 — Opret og afslut forbindelse

```
1 import snap7
2
3 # ===== PLC opsætning =====
4 plc_ip = "192.168.154.1"
5 rack   = 0
6 slot   = 1
7 port   = 1102
8
9 # ===== PLC forbindelse =====
10 client = snap7.client.Client()
11 client.connect(plc_ip, rack, slot, port)
12 print("Forbundet til PLC:", client.get_cpu_info())
13
14 # ===== Disconnect og destroy ===
15 client.disconnect()
16 client.destroy()
```

God praksis

Kald **altid** `client.disconnect()` og `client.destroy()` til sidst. Ellers kan PLC'en komme til at bruge resurser på en hængende forbindelse.

3 | Læse data fra PLC (db_read)

Data i en Siemens PLC er organiseret i **Data Blocks (DB)**. Hver variabel har en byte-offset og en størrelse. Nedenstående eksempel læser tre datatyper fra DB1:

Variabel	Offset	Størrelse	Type	TIA Portal navn
Bool	0	1 byte	DBX0.0	DB1.DBX0.0
Int	2	2 bytes	DBW2	DB1.DBW2
Real	4	4 bytes	DBD4	DB1.DBD4

Kode 2 — Læse Bool, Int og Real fra DB1

```

1 import snap7
2 from snap7.util import get_bool, get_int, get_real
3
4 # ===== PLC opsætning =====
5 plc_ip      = "192.168.154.1"
6 rack       = 0
7 slot       = 1
8 port       = 1102
9 db_number  = 1
10 bool_offset = 0; bool_size = 1
11 int_offset  = 2; int_size  = 2
12 real_offset = 4; real_size = 4
13
14 # ===== PLC forbindelse =====
15 client = snap7.client.Client()
16 client.connect(plc_ip, rack, slot, port)
17
18 # ===== Læs input =====
19 raw_bool = client.db_read(db_number, bool_offset, bool_size)
20 raw_int  = client.db_read(db_number, int_offset, int_size)
21 raw_real = client.db_read(db_number, real_offset, real_size)
22
23 my_bool = get_bool(raw_bool, 0, 0)    # DB1.DBX0.0
24 my_int  = get_int(raw_int, 0)         # DB1.DBW2
25 my_real = get_real(raw_real, 0)       # DB1.DBD4
26
27 # ===== Logik =====
28 print("Bool:", my_bool)
29 print("Int:", my_int)
30 print("Real:", round(my_real, 2))
31
32 # ===== Disconnect og destroy ===
33 client.disconnect()
34 client.destroy()

```

4 | Skrive data til PLC (db_write)

For at **skrive** til PLC'en bruger vi `set_bool`, `set_int` og `set_real` til at modificere en byte-buffer, som derefter sendes med `client.db_write`.

Kode 3 — Skrive Bool, Int og Real til DB1

```
1 import snap7
2 from snap7.util import get_bool, get_int, get_real
3
4 # ===== PLC opsætning =====
5 plc_ip      = "192.168.154.120"
6 rack       = 0
7 slot       = 1
8 db_number   = 1
9 bool_offset = 0; bool_size = 1
10 int_offset  = 2; int_size  = 2
11 real_offset = 4; real_size = 4
12
13 # ===== PLC forbindelse =====
14 client = snap7.client.Client()
15 client.connect(plc_ip, rack, slot)
16
17 # ===== Læs input =====
18 raw_bool = client.db_read(db_number, bool_offset, bool_size)
19 raw_int  = client.db_read(db_number, int_offset, int_size)
20 raw_real = client.db_read(db_number, real_offset, real_size)
21
22 # ===== Logik =====
23 set_bool(raw_bool, bool_offset, 0, False)
24 set_int(raw_int, 0, 20)
25 set_real(raw_real, 0, 111.2)
26
27 # ===== Skriv output =====
28 client.db_write(db_number, bool_offset, raw_bool)
29 client.db_write(db_number, int_offset, raw_int)
30 client.db_write(db_number, real_offset, raw_real)
31
32 print("Bool skrevet:", get_bool(client.db_read(db_number, 0, 1), 0, 0))
33 print("Int skrevet:", get_int(client.db_read(db_number, int_offset,
34   ↪ int_size), 0))
35 print("Real skrevet:", round(get_real(client.db_read(db_number,
36   ↪ real_offset, real_size), 0), 2))
37
38 # ===== Disconnect og destroy ===
39 client.disconnect()
40 client.destroy()
```

5 | Betingede handlinger (if-sætning)

I dette eksempel læser vi en Bool fra PLC'en og bruger dens tilstand til at bestemme hvilken Int-værdi der skrives tilbage.

Kode 4 — If-sætning baseret på PLC-data

```
1 import snap7
2 from snap7.util import get_bool, get_int, set_int
3
4 # ===== PLC opsætning =====
5 PLC_IP      = "192.168.154.120"
6 RACK        = 0
7 SLOT        = 1
8 DB_NUMBER   = 1
9 BOOL_OFFSET = 0;  BOOL_SIZE = 1
10 INT_OFFSET  = 2;  INT_SIZE  = 2
11
12 # ===== PLC forbindelse =====
13 client = snap7.client.Client()
14 client.connect(PLC_IP, RACK, SLOT)
15
16 # ===== Læs input =====
17 raw_bool = client.db_read(DB_NUMBER, BOOL_OFFSET, BOOL_SIZE)
18 raw_int  = client.db_read(DB_NUMBER, INT_OFFSET, INT_SIZE)
19 my_bool  = get_bool(raw_bool, 0, 0)
20 my_int   = get_int(raw_int, 0)
21
22 # ===== Logik =====
23 if my_bool == False:
24     set_int(raw_int, 0, 35)
25 else:
26     set_int(raw_int, 0, 88)
27
28 # ===== Skriv output =====
29 client.db_write(DB_NUMBER, INT_OFFSET, raw_int)
30 my_int = get_int(client.db_read(DB_NUMBER, INT_OFFSET, INT_SIZE), 0)
31 print("my_int =", my_int)
32
33 # ===== Disconnect og destroy ===
34 client.disconnect()
35 client.destroy()
```

6 | Fejlhåndtering (try / except / finally)

I produktionskode er det vigtigt at håndtere forbindelsesfejl. Python's try/except/finally sikrer at forbindelsen *altid* lukkes, selv hvis der opstår en fejl undervejs.

Kode 5 — Robust fejlhåndtering

```
1 import snap7
2 from snap7.util import get_bool, get_int, get_real
3
4 # ===== PLC opsætning =====
5 plc_ip      = "192.168.154.12"
6 rack        = 0
7 slot        = 1
8 db_number   = 1
9 bool_offset = 0; bool_size = 1
10 int_offset  = 2; int_size  = 2
11 real_offset = 4; real_size = 4
12
13 try:
14     # ===== PLC forbindelse =====
15     client = snap7.client.Client()
16     client.connect(plc_ip, rack, slot)
17
18     # ===== Læs input =====
19     raw_bool = client.db_read(db_number, bool_offset, bool_size)
20     raw_int  = client.db_read(db_number, int_offset, int_size)
21     raw_real = client.db_read(db_number, real_offset, real_size)
22
23     # ===== Logik =====
24     set_bool(raw_bool, bool_offset, 0, False)
25     set_int(raw_int, 0, 20)
26     set_real(raw_real, 0, 111.2)
27     print("Alt gik godt!")
28
29     # ===== Skriv output =====
30     client.db_write(db_number, bool_offset, raw_bool)
31     client.db_write(db_number, int_offset, raw_int)
32     client.db_write(db_number, real_offset, raw_real)
33
34 except Exception as e:
35     print(f"Fejl: Kunne ikke forbinde til PLC - {e}")
36
37 finally:
38     # ===== Disconnect og destroy ===
39     client.disconnect()
40     client.destroy()
41     print("Forbindelsen er lukket og klient-objekt er destrueret.")
```


**i try / except / finally forklaret**

try	Kode der <i>kan</i> fejle køres her.
except	Køres <i>kun</i> hvis der opstod en fejl i try -blokken.
finally	Køres altid — uanset om der var fejl eller ej. Bruges til oprydning (disconnect/destroy).

7 | Oversigt over snap7 funktioner

Funktion	Beskrivelse	Modul
<code>client.connect(ip, rack, slot)</code>	Opretter forbindelse til PLC	snap7
<code>client.disconnect()</code>	Lukker forbindelsen	snap7
<code>client.destroy()</code>	Destruerer klient-objektet	snap7
<code>client.db_read(db, offset, size)</code>	Læser bytes fra en DB	snap7
<code>client.db_write(db, offset, data)</code>	Skriver bytes til en DB	snap7
<code>get_bool(data, byte, bit)</code>	Udtrækker en Bool-værdi	snap7.util
<code>get_int(data, byte)</code>	Udtrækker en Int-værdi	snap7.util
<code>get_real(data, byte)</code>	Udtrækker en Real-værdi	snap7.util
<code>set_bool(data, byte, bit, val)</code>	Sætter en Bool-værdi	snap7.util
<code>set_int(data, byte, val)</code>	Sætter en Int-værdi	snap7.util
<code>set_real(data, byte, val)</code>	Sætter en Real-værdi	snap7.util

8 | Øvelse

☰ Øvelsesopgaver

1. Opret forbindelse til den udleverede Siemens PLC med din egen IP-adresse.
2. Læs Bool-, Int- og Real-værdier fra DB1 og udskriv dem.
3. Skriv to forskellige Int-værdier til DB1 baseret på Bool-værdien (brug `if/else`).
4. Implementer fejlhåndtering med `try/except/finally` i dit program.
5. **Ekstra:** Lav en løkke der kontinuerligt læser Real-værdien og udskriver den hvert sekund.